

Lecture 5: Link Layer

Error Control Coding

EC 441 - Introduction to Computer Networking
Boston University

Spring 2026

Learning Objectives

By the end of this lecture, you should be able to:

- Explain why error control is needed at the link layer
- Define block codes using (n, k) notation and compute the code rate
- Distinguish between error detection, error correction, and combined approaches
- Compute the Hamming distance between codewords and determine $d_{\min}$ for a code
- Apply the $d_{\min}$ theorem to determine valid (e_c, e_d) pairs
- Analyze design tradeoffs between error detection and error correction

Error Control Coding

Goal: Decrease the residual bit error rate (BER) without reducing throughput too much

Key Idea: Add redundant bits to allow detection and/or correction of errors

Fundamental Tradeoff:

- Increasing redundant bits improves BER
- But decreases throughput

Where it fits in the stack:

Application → Transport → Network → Link: Error Control → Physical

Block Codes: (n, k) Notation

Error-Control Coding Framework:

- Take k **data bits** (Data), transmit an n -bit **codeword** (Word)
- There are 2^k valid codewords out of 2^n possible n -bit words
- The remaining $2^n - 2^k$ words are **non-codewords** (invalid)
- This is called an (n, k) **block code**

Encoding: Calculate $\text{Word} = g(\text{Data})$ and transmit Word

Common Structure:

$$\text{Word} = \text{Data} \mid f(\text{Data})$$

i.e., send original data plus $n - k$ check bits

Rate of the code:

$$R_c = \frac{k}{n}$$

- $R_c = 1$: No redundancy (no error protection)
- $R_c < 1$: Redundancy added (throughput reduced by factor R_c)
- Lower R_c means more redundancy $\Rightarrow$ potentially better error protection

Decoding: At receiver, calculate $\text{Data} = g^{-1}(\text{Received Word})$

Error Control Options

1. Error Correction (FEC):

- Any received word is mapped to the nearest valid codeword
- No retransmission needed
- Higher overhead (more redundant bits)

2. Error Detection (ARQ):

- Only the 2^k valid codewords are accepted
- Invalid words trigger a retransmission request
- Lower overhead, but requires a feedback channel

3. Combined Detection and Correction:

- Some non-codewords are corrected (close to a valid codeword)
- Some non-codewords are rejected (not close to any valid codeword)
- All valid codewords are accepted

Example: Parity Bit Code (Even Parity)

Here $n - k = 1$: send k data bits followed by a parity bit forcing an **even number of ones**

Data (3 bits)	Parity Bit	Codeword (4 bits)
000	0	0000
001	1	0011
010	1	0101
011	0	0110
100	1	1001
101	0	1010
110	0	1100
111	1	1111

Properties: Rate $R_c = k/(k + 1)$ (efficient). Detects any odd number of errors. Cannot correct errors.

Example: Repetition Code

Here $k = 1$ and n can be chosen as needed

Two codewords: n zeros or n ones

For $n = 3$ repetition code:

- Codewords: $\{000, 111\}$
- Rate: $R_c = 1/3$ (very inefficient)
- $d_{\min} = 3$

Options:

- Can detect up to 2 errors ($e_c = 0, e_d = 2$)
- **OR** correct 1 error ($e_c = 1, e_d = 1$)

General repetition by n : Rate $= 1/n$, $d_{\min} = n$

Hamming Distance

Definition: The **Hamming distance** between two words is the number of bit positions in which they differ.

Example:

$$\text{dist}(0011, 1010) = 2$$

since they differ in the 1st and 4th positions.

Interpretation: Think of codewords as points in an n -dimensional binary space. Hamming distance is the “city block” distance in this space.

Minimum Distance of a Code

A **code** is a set $\mathcal{C}$ of n -bit codewords.

Minimum distance $d_{\min}$ is the smallest Hamming distance between any pair of distinct codewords:

$$d_{\min} = \min_{C_i \neq C_j \in \mathcal{C}} \text{dist}(C_i, C_j)$$

Why $d_{\min}$ matters:

- It **completely determines** the error detection and correction capability
- Larger $d_{\min} \Rightarrow$ more powerful code
- But typically requires more redundancy (lower R_c)

Computing $d_{\min}$: Example

Consider the code $\mathcal{C} = \{000, 011, 101, 110\}$:

Pair	Distance
dist(000, 011)	2
dist(000, 101)	2
dist(000, 110)	2
dist(011, 101)	2
dist(011, 110)	2
dist(101, 110)	2

Therefore $d_{\min} = 2$.

Note: $\binom{4}{2} = 6$ pairs to check for 4 codewords. In general, $\binom{|\mathcal{C}|}{2}$ pairs.

Code Power: e_c and e_d

Define:

- e_c = number of errors **guaranteed to be corrected** successfully
- e_d = number of errors **guaranteed to be detected** successfully

Requirements:

- $e_c \leq e_d$ (if we can correct, we must first detect)
- The pair (e_c, e_d) describes the receiver's error control strategy

The key question: Given a code with known $d_{\min}$, what (e_c, e_d) pairs are valid?

Receiver Algorithm

Let e be the apparent number of errors in received word R :

$$e = \min_i \text{dist}(R, C_i)$$

Decision rule:

- If $0 \leq e \leq e_c$: **Correct** – pick C_i as the intended codeword
- If $e_c < e$: **Detect** – request retransmission, discard, or flag as uncorrectable

The algorithm is fully described by:

- 1 The code $\mathcal{C}$ (set of valid codewords)
- 2 The parameters e_c and e_d

Hamming Sphere Interpretation

Visualize each codeword with a “sphere” of radius e_c :

- All words within distance e_c of codeword C_i are **corrected** to C_i
- Words outside all spheres but within distance e_d of the nearest codeword are **detected** as errors
- Words farther than e_d from all codewords: undetectable errors

Three configurations:

- **Combined correction + detection:** Small spheres, gap between them
- **Maximum correction:** Spheres as large as possible (touching)
- **Maximum detection:** No spheres ($e_c = 0$), maximum gap

The $d_{\min}$ Theorem

Theorem. For a code with minimum distance $d_{\min}$, the receiver uses an (e_c, e_d) pair that must satisfy:

$$e_c + e_d \leq d_{\min} - 1$$

$$e_c \leq e_d$$

This theorem governs **all** error control code design.

$d_{\min}$ Theorem: Proof Sketch

Why $e_c \leq e_d$?

If we can correct an error, we must necessarily detect its presence first. Hence $e_c \leq e_d$.

Why $e_c + e_d \leq d_{\min} - 1$?

Suppose $e_c + e_d = d_{\min}$. Consider the two closest codewords C_i and C_j with $\text{dist}(C_i, C_j) = d_{\min}$.

There exists a received word R that is:

- Distance e_c from C_i
- Distance e_d from C_j

Contradiction:

- If C_i was sent $\Rightarrow$ we should **correct** R to C_i
- If C_j was sent $\Rightarrow$ we should **detect** the error and reject R
- Algorithm must choose one action $\Rightarrow$ cannot guarantee **both**

Therefore $e_c + e_d < d_{\min}$, i.e., $e_c + e_d \leq d_{\min} - 1$.

Consequences of the $d_{\min}$ Theorem

(1) **Maximum error detection** (set $e_c = 0$):

$$(e_d)_{\max} = d_{\min} - 1$$

(2) **Maximum error correction** (set $e_c = e_d$):

$$2e_c \leq d_{\min} - 1 \implies (e_c)_{\max} = \left\lfloor \frac{d_{\min} - 1}{2} \right\rfloor$$

(3) **You cannot simultaneously maximize both:**

- Cannot achieve $\left(\left\lfloor \frac{d_{\min}-1}{2} \right\rfloor, d_{\min} - 1 \right)$
- Must **choose** between best detection and best correction

$d_{\min}$ Example: $d_{\min} = 7$

e_c	e_d	Note
0	6	$(e_d)_{\max} = 6$ (best detection)
1	5	
2	4	
3	3	$(e_c)_{\max} = 3$ (best correction)

Observation: Each unit of correction “costs” one unit of detection.

Verify: $e_c + e_d \leq 6$ and $e_c \leq e_d$ in every row.

Quick Reference: Choosing $d_{\min}$

To detect d errors:

$$d_{\min} = d + 1$$

To correct d errors:

$$d_{\min} = 2d + 1$$

Examples:

- Detect 1 error $\Rightarrow d_{\min} = 2$ (parity bit achieves this)
- Correct 1 error $\Rightarrow d_{\min} = 3$ (repetition-3 achieves this)
- Detect 2 errors $\Rightarrow d_{\min} = 3$
- Correct 2 errors $\Rightarrow d_{\min} = 5$

Design Example 1: Problem

Problem: Design a code from 3-bit codewords with 2 codewords.

Questions:

- 1 What is the rate of the code?
- 2 Which codewords should you send?
- 3 What options exist for error detecting or correcting?

Design Example 1: Solution

Rate: $R_c = k/n = 1/3$

Codewords: Pick any two “opposites”: $\{000, 111\}$ are convenient. Then $d_{\min} = 3$.

Two sensible options:

Option	(e_c, e_d)	Decoding Rule
Error correction	(1, 1)	$\{000, 100, 010, 001\} \rightarrow 0$ $\{111, 110, 101, 011\} \rightarrow 1$
Error detection	(0, 2)	$000 \rightarrow 0; 111 \rightarrow 1$ All others $\rightarrow$ retransmit

Design Example 2: Problem

Problem: Design a code using 3-bit codewords with 4 codewords.

Questions:

- ① What is the rate of the code?
- ② Which codewords should you send?
- ③ What options exist for error detecting or correcting?

Design Example 2: Solution

Rate: $R_c = k/n = 2/3$

Codewords: $\{000, 011, 101, 110\}$. Then $d_{\min} = 2$.

Only one sensible option: Error detection with ($e_c = 0, e_d = 1$)

Codeword	Data
000	00
011	01
101	10
110	11
Others	Retransmit

With $d_{\min} = 2$: can detect 1 error, cannot correct any errors.

When to Use Error Correction (FEC)

Error detection with ARQ is usually $3\text{--}5\times$ more efficient for typical networking

Use FEC when retransmission is impossible or impractical:

Scenario	Reason
Storage (CDs, hard drives, tape)	Errors are permanent
Real-time (voice, video)	No time for retransmission
Simplex (digital broadcast TV)	No return path
High-delay (satellite, deep space)	RTT makes retransmission too slow

Key Takeaways

① Block Codes

- Add $n - k$ redundant bits to k data bits, creating n -bit codewords
- Word = Data | $f(\text{Data})$
- Code rate $R_c = k/n$ measures efficiency

② Hamming Distance and $d_{\min}$

- Hamming distance counts differing bit positions
- $d_{\min}$ = minimum pairwise distance among all codewords
- $d_{\min}$ completely determines error control capability

③ Error Control Options

- Correction (FEC): map to nearest codeword
- Detection (ARQ): reject invalid words, retransmit
- Combined: correct some errors, detect others

Key Takeaways (continued)

4 The $d_{\min}$ Theorem

- $e_c + e_d \leq d_{\min} - 1$ with $e_c \leq e_d$
- Best detection: $(0, d_{\min} - 1)$
- Best correction: $(\lfloor \frac{d_{\min}-1}{2} \rfloor, \lfloor \frac{d_{\min}-1}{2} \rfloor)$
- Cannot simultaneously maximize both

5 Quick Reference

- Detect d errors $\Rightarrow d_{\min} = d + 1$
- Correct d errors $\Rightarrow d_{\min} = 2d + 1$

6 Design Choice

- Detection (ARQ) is usually more efficient for networking
- Correction (FEC) is essential when retransmission is impossible

Error control is fundamental to the link layer:

- Every Ethernet frame uses CRC error detection
- WiFi uses both FEC and CRC
- TCP/IP checksums provide end-to-end error detection
- Storage systems (RAID, SSDs) rely heavily on error correction

Next Lecture (Lecture 6): CRC Codes

- Single-bit error correction theorem
- Quantitative comparison: detection vs. correction vs. repetition
- CRC (Cyclic Redundancy Check) codes – the workhorses of networking
- Generator polynomials and CRC computation